Sreyas Institute of Engineering and Technology
An Autonomous Institution
Approved by AICTE, Affiliated to JNTUH
Accredited by NAAC-A Grade, NBA (CSE, ECE & ME) & ISO 9001:2015 Certified

DEVOPS LAB MANUAL

B.Tech IV Year – I Semester

Department of Computer Science and Engineering

Regulation : R22

Academic Year : 2026 – 2027

Course	DEVOPS LAB
Programme	B.Tech – Computer Science and Engineering
Institution	Sreyas Institute of Engineering and Technology, Hyderabad
Regulation	R22

PREPARED BY	VERIFIED BY
Nitin Kumar Assistant Professor Department of Computer Science and Engineering Employee ID: 1089 Sreyas Institute of Engineering and Technology, Hyderabad Signature: Nitin Kumar	Dr. U.M. FERNANDES DIMLO Head of the Department (CSE) Sreyas Institute of Engineering and Technology, Hyderabad Signature:

Date: 30 June 2026

SREYAS INSTITUTE OF ENGINEERING AND TECHNOLOGY

Hyderabad, Telangana

Excellence • Innovation • Integrity • Service

DEVOPS LAB MANUAL

Experiment No. 1
Title: Simple Event Registration Form using HTML and Selenium Automation

1. Aim of the Experiment

To develop a simple Event Registration Form using HTML and automate its testing using Selenium WebDriver with Python.

2. Objectives

- Design a basic HTML registration form.
- Understand Selenium WebDriver.
- Automate browser interactions.
- Verify application output automatically.
- Understand the role of automation testing in DevOps.

3. Apparatus / Software Required

- Windows/Linux OS
- Python 3.x
- Google Chrome
- ChromeDriver
- Visual Studio Code
- Selenium Library
- Live Server Extension

4. Theory

DevOps integrates software development and IT operations. Continuous Testing is a core DevOps practice. Selenium is an open-source automation framework used to automate browser actions such as entering data, clicking buttons and validating results. It supports CI/CD pipelines and reduces manual testing effort.

5. Algorithm

Step 1: Create registration.html.

Step 2: Run the page using Live Server.

Step 3: Install Selenium (pip install selenium).

Step 4: Launch Chrome using Selenium.

Step 5: Open the registration page.

Step 6: Enter Name, Email and Mobile.

Step 7: Click Register.

Step 8: Read success message.

Step 9: Compare expected and actual output.

Step 10: Close browser.

6. HTML Program

```
<!DOCTYPE html>
<html>
<head><title>Event Registration</title></head>
<body>
<h2>Event Registration Form</h2>
<label>Name:</label><input type="text" id="name"><br><br>
<label>Email:</label><input type="text" id="email"><br><br>
<label>Mobile:</label><input type="text" id="mobile"><br><br>
<button onclick="register()">Register</button>
<p id="message"></p>
<script>
function register(){
document.getElementById("message").innerHTML="Registration
Successful!";
}
</script>
</body>
</html>
```

---

7. Selenium Python Program

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
```

```
driver.get("http://127.0.0.1:5500/registration.html")
time.sleep(2)
driver.find_element(By.ID,"name").send_keys("Nitin Kumar")
driver.find_element(By.ID,"email").send_keys("nitin@gmail.com")
driver.find_element(By.ID,"mobile").send_keys("9876543210")
driver.find_element(By.TAG_NAME,"button").click()
time.sleep(2)
message = driver.find_element(By.ID,"message").text
print("Result:", message)
if message=="Registration Successful!":
   print("Test Passed")
else:
   print("Test Failed")
driver.quit()
```

---

8. Program Explanation

Statement	Purpose
webdriver.Chrome()	Launches Chrome browser
driver.get()	Opens registration page
find_element()	Locates HTML element
send_keys()	Enters text
click()	Clicks Register button
Text	Reads success message
driver.quit()	Closes browser

9. Sample Input

Name: Nitin Kumar
Email: nitin@gmail.com
Mobile: 9876543210

10. Expected Output

Result: Registration Successful!
Test Passed

11. Real-Time Applications

- IRCTC: Registration and login validation before releases.
- Amazon: Automated customer registration testing in CI/CD.
- Flipkart: Regression testing of registration and checkout workflows.
- College event registration portals.

12. Advantages

- Faster testing
- Repeatable execution
- Reduces manual effort
- Improves software quality
- Supports DevOps CI/CD

13. Limitations

- Browser driver dependency
- UI changes require script updates
- Limited to web UI testing

14. Viva Questions

1. What is Selenium?
2. What is WebDriver?
3. Why is Selenium used in DevOps?
4. What is ChromeDriver?
5. What is Continuous Testing?
6. What is By.ID?
7. Difference between manual and automation testing?
8. Why use driver.quit()?

15. Examination Questions

- Define Selenium, WebDriver, Automation Testing.
- Explain Selenium architecture and locators.
- Explain automation testing using Selenium with suitable example.

16. Result

The Event Registration Form was successfully created using HTML and tested automatically using Selenium WebDriver. The expected success message was displayed and the test passed.

DEVOPS LAB MANUAL

Experiment No. 2
Title: Explore Git and GitHub Commands

1. Aim of the Experiment

To understand the basics of Git and GitHub by creating a local repository, performing version control operations, and synchronizing code with a remote GitHub repository.

2. Objectives

- Understand distributed version control.
- Install and configure Git.
- Create a local Git repository.
- Track file changes using commits.
- Create a GitHub repository.
- Push and pull code between local and remote repositories.

3. Apparatus / Software Required

- Windows/Linux OS
- Git
- GitHub Account
- Visual Studio Code or Git Bash
- Internet Connection

4. Prerequisites

- Basic command-line knowledge
- GitHub account
- Git installed

5. Theory

Git is a distributed version control system used to track source code changes. GitHub is a cloud-based hosting platform for Git repositories, enabling collaboration, code reviews, issue tracking, and CI/CD integration. Git is a foundational DevOps tool because it supports collaboration, traceability, and automated software delivery.

6. Real-Time Applications

- Amazon: Thousands of developers collaborate using Git-based workflows integrated with CI/CD.
- Flipkart: Feature branches and pull requests are reviewed before production deployment.
- IRCTC: Version control helps safely manage ticket-booking application updates.
- Open-source projects: GitHub enables distributed collaboration across the world.

7. Algorithm

Step 1: Install Git.

Step 2: Configure username and email.

Step 3: Create a project folder.

Step 4: Initialize a Git repository.

Step 5: Create a sample file.

Step 6: Add files to staging area.

Step 7: Commit changes.

Step 8: Create a GitHub repository.

Step 9: Connect local repository to GitHub.

Step 10: Push commits.

Step 11: Modify a file and commit again.

Step 12: Pull latest changes.

8. Common Git Commands

Command	Purpose
git --version	Display Git version
git config --global user.name "Your Name"	Configure username
git config --global user.email "you@example.com"	Configure email
git init	Initialize repository
git status	Show repository status
git add .	Stage all files
git commit -m "Initial commit"	Commit staged changes

git log	View commit history
git branch	List branches
git checkout -b feature	Create and switch branch
git merge feature	Merge branch
git remote add origin <repo-url>	Add remote repository
git push -u origin main	Push to GitHub
git pull origin main	Pull latest changes
git clone <repo-url>	Clone repository

9. Sample Procedure

1. Create a folder named GitLabDemo.
2. Run 'git init'.
3. Create README.md.
4. Run 'git add .'.
5. Commit using 'git commit -m "Initial commit"'.
6. Create a repository on GitHub.
7. Add remote origin and push the code.

10. Expected Output

- Local Git repository initialized.
- Commit created successfully.
- Repository uploaded to GitHub.
- Git log displays commit history.

11. Advantages

- Tracks code history
- Supports collaboration
- Enables rollback
- Integrates with CI/CD
- Branching and merging support

12. Limitations

- Learning curve for beginners
- Merge conflicts require resolution
- Remote operations require internet

13. Viva Questions

1. What is Git?
2. What is GitHub?
3. Difference between Git and GitHub?
4. What is a repository?
5. What is a commit?
6. What is staging?
7. What is branching?
8. What is merging?
9. What is clone?
10. What is pull request?

14. Examination Questions

- Define Git.
- Define Repository.
- What is GitHub?
- Explain Git workflow.
- Explain branching and merging.
- Explain Git architecture and common commands with examples.
- Explain how GitHub supports collaborative software development.

15. Result

Git was successfully configured, a local repository was created, files were committed, and the repository was synchronized with GitHub using push and pull operations.

DEVOPS LAB MANUAL

Experiment No. 3
Title: Practice Source Code Management on GitHub using the Event Registration Application

1. Aim of the Experiment
To practice Source Code Management (SCM) using GitHub by managing versions of the Event Registration application developed in Experiment 1.

2. Objectives
- Understand Source Code Management (SCM).
- Create and manage a GitHub repository.
- Upload the Event Registration project to GitHub.
- Track code changes using Git commits.
- Create branches and merge changes.
- Practice collaboration using Pull Requests.

3. Apparatus / Software Required
- Windows/Linux OS
- Git
- GitHub Account
- Visual Studio Code
- Git Bash/Terminal
- Internet Connection
- Event Registration project (Experiment 1)

4. Prerequisites
- Git installed and configured.
- GitHub account created.
- Experiment 1 project available locally.

5. Theory
Source Code Management (SCM) is the practice of tracking, controlling, and managing changes to source code. Git is a distributed SCM system, while GitHub provides cloud

hosting for Git repositories, enabling collaboration, code reviews, issue tracking, and CI/CD integration.

6. Real-Time Applications

- Amazon: Development teams work on feature branches that are reviewed before merging.
- Flipkart: Pull requests are used to review changes before production deployment.
- IRCTC: Version control helps safely manage updates to ticket booking and payment modules.
- Open-source projects: Contributors submit pull requests for review on GitHub.

7. Algorithm

Step 1: Open the Experiment 1 project folder.

Step 2: Initialize Git repository if not already initialized.

Step 3: Create a GitHub repository.

Step 4: Add the remote repository URL.

Step 5: Stage project files using git add .

Step 6: Commit changes using git commit.

Step 7: Push code using git push.

Step 8: Create a new branch and modify the project.

Step 9: Commit branch changes.

Step 10: Create a Pull Request on GitHub.

Step 11: Review and merge the Pull Request.

Step 12: Pull the latest changes to the local repository.

8. Important Git Commands

Command	Description
git init	Initialize repository
git status	Show repository status
git add .	Stage files
git commit -m "message"	Commit changes
git branch feature	Create branch

git checkout feature	Switch branch
git merge feature	Merge branch
git remote add origin URL	Add remote
git push -u origin main	Push to GitHub
git pull origin main	Download latest changes
git log	View commit history

9. Practical Task

Modify the Event Registration application created in Experiment 1 by adding an Address field or improving the success message. Commit the changes on a feature branch, push them to GitHub, create a Pull Request, review the changes, and merge them into the main branch.

10. Expected Output

Repository successfully created on GitHub.
Event Registration project uploaded.
Commit history visible.
Feature branch created and merged.
Pull Request completed successfully.

11. Advantages

- Maintains complete code history.
- Supports team collaboration.
- Enables rollback to previous versions.
- Facilitates code review through Pull Requests.
- Integrates with CI/CD pipelines.

12. Limitations

- Requires understanding of Git workflows.
- Merge conflicts may occur.
- Remote operations require internet connectivity.

13. Viva Questions

1. What is SCM?
2. Why is GitHub used?
3. What is a branch?
4. What is a Pull Request?
5. What is merge conflict?
6. Difference between git push and git pull?

7. What is commit history?
8. Why use feature branches?
9. What is code review?
10. How does GitHub support DevOps?

14. Examination Questions

- Define SCM.
- What is GitHub?
- What is a Pull Request?
- Explain GitHub workflow.
- Explain branching strategy.
- Explain Source Code Management using GitHub with suitable commands.
- Explain the complete workflow of managing a project on GitHub using branches and Pull Requests.

15. Result

The Event Registration application from Experiment 1 was successfully managed using GitHub. Source code changes were tracked through commits, synchronized with the remote repository, and integrated using branches and Pull Requests.

DEVOPS LAB MANUAL

Experiment No. 4
Title: Jenkins Installation, Setup and Environment Exploration

1. Aim of the Experiment

To install and configure Jenkins, explore its user interface and environment, and understand its role in Continuous Integration and Continuous Delivery (CI/CD).

2. Objectives

- Install Java and Jenkins.
- Start the Jenkins server.
- Unlock Jenkins and complete the initial setup.
- Install recommended plugins.
- Create the first administrator account.
- Explore the Jenkins dashboard and environment.

3. Apparatus / Software Required

- Windows 10/11 or Ubuntu Linux
- Java JDK 17 or later
- Jenkins LTS
- Google Chrome/Firefox
- Internet connection
- Minimum 4 GB RAM

4. Prerequisites

- Administrator privileges
- Basic command-line knowledge
- Java installed and JAVA_HOME configured

5. Theory

Jenkins is an open-source automation server widely used in DevOps for Continuous Integration (CI) and Continuous Delivery (CD). It automates software builds, testing and deployments using jobs and pipelines. Jenkins supports hundreds of plugins that integrate with Git, GitHub, Docker, Maven, Gradle, Kubernetes and cloud platforms.

6. Real-Time Applications

- Amazon: Jenkins-based pipelines automatically build and test services before deployment.
- Flipkart: CI pipelines validate code before production releases during major sales.
- IRCTC: Automated builds and regression tests reduce deployment risk for booking applications.
- Open-source projects: Jenkins builds every pull request and reports test results.

7. Installation Procedure

Step 1: Install Java JDK and verify using 'java -version'.

Step 2: Download Jenkins LTS from the official website.

Step 3: Run the installer or install via package manager.

Step 4: Start the Jenkins service.

Step 5: Open http://localhost:8080 in a browser.

Step 6: Unlock Jenkins using the initial administrator password.

Step 7: Install recommended plugins.

Step 8: Create the first administrator account.

Step 9: Configure the Jenkins URL and finish setup.

8. Exploring the Jenkins Environment

- Dashboard
- Manage Jenkins
- Manage Plugins
- Global Tool Configuration
- Credentials
- Nodes (Agents)
- New Item (Jobs)
- Build History
- Workspace
- System Information

9. Practical Tasks

1. Verify the Jenkins installation.
2. Create a Freestyle Project.
3. Configure Git as Source Code Management.
4. Execute a simple build.
5. View Console Output.
6. Explore installed plugins and system configuration.

10. Expected Output

• Jenkins dashboard opens successfully.
• Administrator account created.
• Recommended plugins installed.
• A Freestyle job executes successfully.
• Console Output displays 'Build Success'.

11. Advantages

- Automates CI/CD workflows
- Supports hundreds of plugins
- Cross-platform
- Easy integration with Git and Docker
- Scalable using agents

12. Limitations

- Plugin management requires maintenance
- Large installations need adequate hardware
- Security configuration is essential

13. Viva Questions

1. What is Jenkins?
2. Why is Jenkins used in DevOps?
3. What is CI?
4. What is CD?
5. What are Jenkins plugins?
6. What is a Freestyle project?
7. What is a Pipeline?
8. What is a Jenkins Agent?
9. What is the default Jenkins port?
10. How do you unlock Jenkins?

14. Examination Questions

- Define Jenkins.
- What is a Jenkins plugin?
- What is Continuous Integration?
- Explain Jenkins architecture.
- Explain Jenkins installation steps.
- Explain Jenkins installation, configuration and environment with a neat architecture diagram.
- Discuss the role of Jenkins in a CI/CD pipeline.

15. Result

Jenkins was successfully installed and configured. The Jenkins environment was explored, plugins were installed, and a sample build job was created and executed successfully.

DEVOPS LAB MANUAL

Experiment No. 5
Title: Demonstrate Continuous Integration and Continuous Delivery using Jenkins

1. Aim of the Experiment
To demonstrate Continuous Integration (CI) and Continuous Delivery (CD) using Jenkins by integrating a GitHub repository, automating the build process, and verifying successful execution.

2. Objectives
- Understand CI and CD concepts.
- Configure Jenkins with GitHub.
- Create and execute a Jenkins Freestyle or Pipeline job.
- Automate builds after source code changes.
- View build logs and verify build status.

3. Apparatus / Software Required
- Windows/Linux OS
- Java JDK
- Jenkins LTS
- Git
- GitHub Account
- Visual Studio Code
- Google Chrome
- Internet Connection

4. Prerequisites
- Jenkins installed and configured.
- Git installed and configured.
- A GitHub repository with sample source code.
- Git plugin installed in Jenkins.

5. Theory
Continuous Integration (CI) is the practice of automatically integrating code changes from multiple developers into a shared repository. Continuous Delivery (CD) extends CI

by ensuring that validated software is always ready for deployment. Jenkins automates these activities using jobs, pipelines, plugins, and integrations with GitHub.

6. Real-Time Applications

- Amazon: Every code commit automatically triggers builds and tests before deployment.
- Flipkart: CI pipelines validate applications before festival-sale releases.
- IRCTC: Automated builds and regression testing reduce deployment risks for ticket booking services.
- Netflix: Pipeline automation supports frequent and reliable software releases.

7. Algorithm

Step 1: Start Jenkins service.

Step 2: Open Jenkins dashboard (http://localhost:8080).

Step 3: Create a new Freestyle or Pipeline project.

Step 4: Configure GitHub repository URL under Source Code Management.

Step 5: Configure build trigger (Poll SCM or GitHub Webhook).

Step 6: Save the job.

Step 7: Trigger a build manually.

Step 8: Modify source code and push changes to GitHub.

Step 9: Observe automatic build execution.

Step 10: View Console Output and Build History.

8. Practical Procedure

1. Create a Jenkins job.
2. Link the GitHub repository.
3. Configure Git credentials if required.
4. Enable build trigger.
5. Execute the build.
6. Verify build success.
7. Push a code change to GitHub and observe automatic build execution.

9. Sample Jenkins Pipeline

```
pipeline {
    agent any
    stages {
        stage('Checkout'){
            steps { git 'https://github.com/username/sample-repo.git' }
        }
        stage('Build'){
            steps { echo 'Building Application...' }
        }
        stage('Test'){
            steps { echo 'Running Tests...' }
        }
        stage('Deploy'){
            steps { echo 'Ready for Deployment' }
        }
    }
}
```

---

10. Expected Output

• Jenkins successfully connects to GitHub.
• Source code is downloaded.
• Build executes successfully.
• Console Output displays build stages.
• Build status is SUCCESS.

11. Advantages

- Rapid feedback on code changes
- Reduced integration issues
- Improved software quality
- Supports DevOps automation
- Easy integration with GitHub and Docker

12. Limitations

- Requires proper Jenkins configuration
- Plugin compatibility issues may arise
- Infrastructure maintenance is required

13. Viva Questions

1. What is Continuous Integration?
2. What is Continuous Delivery?
3. What is Jenkins Pipeline?
4. What is a Build Trigger?
5. What is a GitHub Webhook?
6. What is Console Output?
7. Difference between Freestyle and Pipeline jobs?
8. What is Poll SCM?
9. Why are plugins required in Jenkins?
10. How does Jenkins support DevOps?

14. Examination Questions

- Define CI.
- Define CD.
- What is Jenkins Pipeline?
- Explain Jenkins build process.
- Explain GitHub integration with Jenkins.
- Explain Continuous Integration and Continuous Delivery using Jenkins with suitable architecture and workflow.

15. Result

The Jenkins server was successfully configured to perform Continuous Integration and Continuous Delivery. A GitHub repository was integrated, builds were automated, and successful build execution was verified through Jenkins.

DEVOPS LAB MANUAL

Experiment No. 6
Title: Explore Docker Commands for Container Management

1. Aim of the Experiment
To install Docker, explore essential Docker commands, and manage Docker images and containers for application deployment in a DevOps environment.

2. Objectives
- Understand Docker architecture.
- Install and verify Docker.
- Pull Docker images from Docker Hub.
- Create, start, stop and remove containers.
- Manage Docker images and volumes.
- Understand Docker's role in CI/CD.

3. Apparatus / Software Required
- Windows 10/11, Ubuntu Linux or macOS
- Docker Desktop or Docker Engine
- Internet Connection
- Terminal/Command Prompt
- Visual Studio Code (optional)
- Minimum 4 GB RAM

4. Prerequisites
- Basic Linux/Command Prompt knowledge
- Docker installed successfully
- Internet access to download images

5. Theory
Docker is an open-source containerization platform that packages an application with its dependencies into lightweight, portable containers. Unlike virtual machines, containers share the host operating system kernel, making them faster and more resource-efficient. Docker is a core DevOps tool because it enables consistent environments across development, testing, and production.

6. Docker Architecture

- Docker Client
- Docker Daemon (dockerd)
- Docker Images
- Docker Containers
- Docker Registry (Docker Hub)
- Docker Volumes
- Docker Networks

7. Real-Time Applications

- Amazon: Uses containers to deploy scalable microservices.
- Flipkart: Packages services into Docker containers for consistent deployments during peak shopping events.
- IRCTC: Containerization helps isolate booking, payment, and notification services.
- Netflix: Uses container technologies for scalable cloud-native applications.

8. Algorithm

Step 1: Install Docker Desktop or Docker Engine.

Step 2: Verify installation using 'docker --version'.

Step 3: Run 'docker info'.

Step 4: Pull the Ubuntu image.

Step 5: List downloaded images.

Step 6: Create and run a container.

Step 7: Execute commands inside the container.

Step 8: Stop and restart the container.

Step 9: Remove the container.

Step 10: Remove unused images.

9. Important Docker Commands

Command	Purpose
docker –version	Display Docker version
docker info	Show Docker environment
docker pull Ubuntu	Download Ubuntu image

docker images	List images
docker run -it Ubuntu	Create and start a container
docker ps	List running containers
docker ps –a	List all containers
docker exec -it <id> bash	Access a running container
docker stop <id>	Stop container
docker start <id>	Start container
docker rm <id>	Remove container
docker rmi <image>	Remove image
docker logs <id>	View logs

10. Practical Tasks

1. Verify Docker installation.
2. Pull the Ubuntu image.
3. Launch an interactive container.
4. Execute Linux commands inside the container.
5. Stop and restart the container.
6. Display running containers and images.
7. Remove the container and image.

11. Expected Output

• Docker installed successfully.
• Ubuntu image downloaded.
• Container created and executed.
• Docker commands executed successfully.
• Images and containers listed correctly.

12. Advantages

- Lightweight and portable
- Fast deployment
- Environment consistency
- Supports microservices
- Easy CI/CD integration

13. Limitations

- Requires shared host kernel
- Persistent storage requires volumes
- Security configuration is important

14. Viva Questions

1. What is Docker?

2. Difference between image and container?

3. What is Docker Hub?

4. What is docker run?

5. What is docker pull?

6. What is docker exec?

7. Difference between VM and container?

8. What are Docker volumes?

9. What is Docker networking?

10. How does Docker support DevOps?

15. Examination Questions

- Define Docker.
- What is a Docker image?
- What is a container?
- Explain Docker architecture.
- Explain important Docker commands.
- Explain Docker architecture and container lifecycle with suitable examples.
- Discuss the role of Docker in DevOps and CI/CD.

16. Result

The Docker environment was successfully explored. Docker images and containers were created, managed, and removed using standard Docker commands, demonstrating container management in DevOps.

DEVOPS LAB MANUAL

Experiment No. 7
Title: Develop a Simple Containerized Application using Docker

1. Aim of the Experiment
To develop a simple web application, containerize it using Docker, build a Docker image, run the application inside a Docker container, and verify its execution.

2. Objectives
- Understand application containerization.
- Create a Dockerfile.
- Build a Docker image.
- Run and manage Docker containers.
- Verify application deployment in a browser.
- Understand Docker's role in DevOps and CI/CD.

3. Apparatus / Software Required
- Windows/Linux/macOS
- Docker Desktop or Docker Engine
- Visual Studio Code
- Web Browser
- Terminal/Command Prompt
- Basic HTML application

4. Prerequisites
- Docker installed successfully.
- Basic knowledge of HTML and Docker commands.
- Internet access (optional, for pulling base images).

5. Theory
A container packages an application together with its runtime, libraries and dependencies, ensuring that it runs consistently across development, testing and production environments. Docker simplifies application deployment by creating lightweight, portable containers.

6. Real-Time Applications

- Amazon packages microservices into containers for scalable cloud deployments.
- Flipkart deploys containerized services during high-traffic shopping events.
- IRCTC isolates booking, payment and notification services using container technology.
- Netflix uses container-based cloud-native deployments for reliability and scalability.

7. Sample Application Structure

project/
├── index.html
├── Dockerfile
└── README.md

8. Sample HTML File (index.html)

```
<!DOCTYPE html>
<html>
<head><title>Docker Demo</title></head>
<body>
<h1>Welcome to Docker</h1>
<p>Containerized Web Application</p>
</body>
</html>
```

9. Sample Dockerfile

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

10. Algorithm

Step 1: Create the project folder.

Step 2: Create index.html.

Step 3: Create a Dockerfile.

Step 4: Build the Docker image using docker build -t docker-demo .

Step 5: Verify the image using docker images.

Step 6: Run the container using docker run -d -p 8080:80 docker-demo.

Step 7: Open http://localhost:8080 in a browser.

Step 8: Verify the output.

Step 9: Stop and remove the container.

11. Important Docker Commands

Command	Purpose
docker build -t docker-demo .	Build image
docker images	List images
docker run -d -p 8080:80 docker-demo	Run container
docker ps	List running containers
docker logs <container>	View logs
docker stop <container>	Stop container
docker rm <container>	Remove container
docker rmi docker-demo	Remove image

12. Expected Output

The Docker image is created successfully, the container starts without errors, and opening http://localhost:8080 displays 'Welcome to Docker'.

13. Advantages

- Portable deployment
- Environment consistency
- Rapid scaling
- Lightweight execution
- Easy CI/CD integration

14. Limitations

- Requires Docker runtime
- Persistent storage needs volumes
- Container security must be configured

15. Viva Questions

1. What is a container?
2. What is a Docker image?
3. What is a Dockerfile?
4. What is docker build?

5. What is docker run?
6. Why expose a port?
7. Difference between VM and container?
8. What is Nginx?
9. What is Docker Hub?
10. How does Docker support DevOps?

16. Examination Questions

- Define Dockerfile.
- What is a container?
- What is a Docker image?
- Explain Docker image creation.
- Explain Docker container lifecycle.
- Develop and deploy a simple containerized application using Docker with suitable commands and architecture.

17. Result

A simple web application was successfully containerized using Docker. The Docker image was built, the container was executed, and the application was accessed successfully through a web browser.

DEVOPS LAB MANUAL

Experiment No. 8
Title: Integrate Kubernetes and Docker

1. Aim of the Experiment
To integrate Docker and Kubernetes by deploying a Docker container as a Kubernetes Pod and managing it using Kubernetes commands.

2. Objectives
- Understand Docker and Kubernetes integration.
- Build or use a Docker image.
- Deploy a containerized application on Kubernetes.
- Create and manage Pods and Deployments.
- Expose the application using a Kubernetes Service.
- Verify application deployment.

3. Apparatus / Software Required
- Windows/Linux/macOS
- Docker Desktop (Kubernetes enabled) or Minikube
- kubectl CLI
- Docker Engine
- Visual Studio Code
- Web Browser
- Internet Connection

4. Prerequisites
- Docker installed and running.
- Kubernetes cluster available (Docker Desktop or Minikube).
- Basic Docker knowledge.
- kubectl configured.

5. Theory
Kubernetes is an open-source container orchestration platform that automates deployment, scaling and management of containerized applications. Docker packages applications into containers, while Kubernetes manages those containers across one or

more machines. Together they form a core part of modern DevOps and cloud-native application delivery.

6. Real-Time Applications

- Amazon: Uses Kubernetes-based orchestration for scalable microservices.
- Flipkart: Deploys containerized services with auto-scaling during festival sales.
- IRCTC: Kubernetes can scale booking services during Tatkal hours.
- Netflix: Uses container orchestration for resilient cloud-native deployments.

7. Sample Kubernetes Deployment (deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: docker-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: docker-demo
  template:
    metadata:
      labels:
        app: docker-demo
    spec:
      containers:
      - name: docker-demo
        image: nginx:latest
        ports:
        - containerPort: 80
```

---

8. Sample Service (service.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: docker-demo-service
spec:
  selector:
```

```
  app: docker-demo
ports:
- port: 80
  targetPort: 80
type: NodePort
```

---

9. Algorithm

Step 1: Start Docker and Kubernetes.

Step 2: Verify cluster using 'kubectl cluster-info'.

Step 3: Build or pull a Docker image.

Step 4: Create deployment.yaml.

Step 5: Apply deployment using 'kubectl apply -f deployment.yaml'.

Step 6: Verify Pods with 'kubectl get pods'.

Step 7: Create service.yaml.

Step 8: Expose application using 'kubectl apply -f service.yaml'.

Step 9: Verify service and access application.

Step 10: Scale deployment and observe replicas.

10. Important Commands

Command	Purpose
kubectl cluster-info	Display cluster information
kubectl get nodes	List nodes
kubectl get pods	List pods
kubectl get deployments	List deployments
kubectl apply -f deployment.yaml	Create deployment
kubectl apply -f service.yaml	Create service
kubectl describe pod <pod>	View pod details
kubectl logs <pod>	View pod logs
kubectl scale deployment docker-demo --replicas=3	Scale deployment
kubectl delete deployment docker-demo	Delete deployment
docker images	List Docker images
docker build -t app .	Build Docker image

11. Expected Output

Docker image is available, Kubernetes deployment is created successfully, Pods are in Running state, the Service exposes the application, and the web page is accessible.

12. Advantages

- Automated deployment
- Auto scaling
- High availability
- Self-healing
- Easy CI/CD integration

13. Limitations

- Steeper learning curve
- Cluster management overhead
- Resource requirements for production clusters

14. Viva Questions

1. What is Kubernetes?

2. What is a Pod?

3. What is a Deployment?

4. What is a Service?

5. Difference between Docker and Kubernetes?

6. What is kubectl?

7. What is NodePort?

8. Why are replicas used?

9. What is container orchestration?

10. How does Kubernetes support DevOps?

15. Examination Questions

- Define Kubernetes.
- What is a Pod?
- What is a Deployment?

- Explain Kubernetes architecture.
- Explain Docker-Kubernetes integration.
- Explain deployment of a Docker container using Kubernetes with suitable commands and architecture.

16. Result

Docker and Kubernetes were successfully integrated. A containerized application was deployed using Kubernetes Deployment and Service resources, and the application was verified successfully.

DEVOPS LAB MANUAL

Experiment No. 9
Title: Automate the Deployment of a Containerized Application using Kubernetes

1. Aim of the Experiment
To automate the deployment and execution of the Docker containerized web application developed in Experiment 7 using Kubernetes Deployments and Services.

2. Objectives
- Understand Kubernetes automation for containerized applications.
- Deploy a Docker image using a Kubernetes Deployment.
- Expose the application through a Kubernetes Service.
- Scale application replicas.
- Monitor Pods and verify automated deployment.

3. Apparatus / Software Required
- Windows/Linux/macOS
- Docker Desktop (Kubernetes enabled) or Minikube
- kubectl CLI
- Docker Engine
- Visual Studio Code
- Web Browser
- Containerized application from Experiment 7

4. Prerequisites
- Experiment 7 completed successfully.
- Docker image available locally or in Docker Hub.
- Kubernetes cluster running.
- kubectl configured.

5. Theory
Kubernetes automates deployment, scaling and management of containerized applications. A Deployment ensures the desired number of Pods are running, while a Service provides a stable endpoint. Automation minimizes manual intervention and supports reliable Continuous Delivery.

6. Real-Time Applications

- Amazon automatically deploys microservices across Kubernetes clusters.
- Flipkart scales application replicas automatically during Big Billion Days.
- IRCTC can increase booking-service replicas during Tatkal hours.
- Netflix uses automated orchestration for resilient cloud-native deployments.

7. Kubernetes Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: event-registration
spec:
  replicas: 2
  selector:
    matchLabels:
      app: event-registration
  template:
    metadata:
      labels:
        app: event-registration
    spec:
      containers:
      - name: event-registration
        image: docker-demo:latest
        ports:
        - containerPort: 80
```

---

8. Kubernetes Service YAML

```
apiVersion: v1
kind: Service
metadata:
  name: event-registration-service
spec:
  selector:
    app: event-registration
  ports:
  - port: 80
```

targetPort: 80
type: NodePort

---

9. Algorithm

Step 1: Start Docker and Kubernetes cluster.

Step 2: Verify cluster using kubectl cluster-info.

Step 3: Ensure Docker image from Experiment 7 is available.

Step 4: Create deployment.yaml and service.yaml.

Step 5: Deploy resources using kubectl apply.

Step 6: Verify Pods and Deployment.

Step 7: Access the application through the Service.

Step 8: Scale replicas using kubectl scale.

Step 9: View logs and monitor Pods.

Step 10: Delete resources after verification.

10. Important Commands

Command	Purpose
kubectl apply -f deployment.yaml	Create Deployment
kubectl apply -f service.yaml	Create Service
kubectl get pods	View Pods
kubectl get deployments	View Deployments
kubectl get services	View Services
kubectl describe pod <pod>	Inspect Pod
kubectl logs <pod>	View logs
kubectl scale deployment event-registration --replicas=3	Scale application
kubectl delete -f deployment.yaml	Delete Deployment
kubectl delete -f service.yaml	Delete Service

11. Expected Output

The application is deployed automatically, Pods are in the Running state, the Service exposes the application successfully, and scaling creates additional replicas without manual redeployment.

12. Advantages

- Automated deployment
- Self-healing
- Auto-scaling
- High availability
- Supports CI/CD

13. Limitations

- Requires Kubernetes cluster
- Cluster administration complexity
- Additional resource consumption

14. Viva Questions

1. What is Kubernetes Deployment?

2. What is a Service?

3. What is a Pod?

4. How does Kubernetes automate deployments?

5. What is replica scaling?

6. What is kubectl apply?

7. Why is NodePort used?

8. What is self-healing?

9. Difference between Docker and Kubernetes?

10. How does Kubernetes support DevOps?

15. Examination Questions

- Define Deployment.
- Define Service.
- What is a Pod?
- Explain Kubernetes Deployment.
- Explain Service types.
- Explain the automation of a Docker containerized application using Kubernetes with suitable commands and architecture.

16. Result

The containerized application developed in Experiment 7 was successfully automated using Kubernetes Deployment and Service resources. The application was deployed, accessed, monitored and scaled successfully.

DEVOPS LAB MANUAL

Experiment No. 10
Title: Install and Explore Selenium for Automated Testing

1. Aim of the Experiment
To install Selenium WebDriver, configure the testing environment, execute a simple automated web test, and explore Selenium features for web application testing.

2. Objectives
- Install Python and Selenium.
- Configure Chrome browser and ChromeDriver.
- Write and execute a basic Selenium script.
- Understand Selenium WebDriver architecture.
- Explore Selenium locators and browser automation.
- Understand Selenium's role in DevOps and CI/CD.

3. Apparatus / Software Required
- Windows/Linux/macOS
- Python 3.x
- Google Chrome Browser
- ChromeDriver (or Selenium Manager)
- Visual Studio Code / PyCharm
- Selenium Python Package
- Internet Connection

4. Prerequisites
- Python installed successfully.
- pip package manager available.
- Basic Python programming knowledge.
- Google Chrome installed.

5. Theory
Selenium is an open-source framework used for automating web browsers. It supports multiple programming languages such as Python, Java, C#, JavaScript and Ruby. Selenium WebDriver directly controls browsers, enabling automated functional,

regression, and cross-browser testing. In DevOps, Selenium is widely integrated with Jenkins and CI/CD pipelines for Continuous Testing.

6. Selenium Components

- Selenium IDE
- Selenium WebDriver
- Selenium Grid
- Selenium Manager
- Browser Drivers

7. Real-Time Applications

- Amazon: Automates testing of product search, shopping cart, checkout and payment pages.
- Flipkart: Executes regression tests before festival sales such as Big Billion Days.
- IRCTC: Validates login, ticket booking and payment workflows before deployment.
- Banking Applications: Automates testing of internet banking and online payment systems.

8. Installation Procedure

Step 1: Install Python from python.org.

Step 2: Verify installation using 'python --version'.

Step 3: Install Selenium using 'pip install selenium'.

Step 4: Install Google Chrome.

Step 5: Verify Selenium installation.

Step 6: Create a Python file (test.py).

Step 7: Write a simple Selenium script.

Step 8: Execute the script using 'python test.py'.

Step 9: Observe browser automation.

9. Sample Selenium Program

```
from selenium import webdriver
from selenium.webdriver.common.by import By
```

```
driver = webdriver.Chrome()
driver.get("https://example.com")
print(driver.title)
driver.quit()
```

---

10. Important Selenium Commands

Command / Method	Purpose
webdriver.Chrome()	Launch Chrome browser
driver.get(url)	Open webpage
find_element(By.ID, value)	Locate web element
send_keys()	Enter text
click()	Click button
driver.title	Read page title
driver.current_url	Display current URL
driver.back()	Navigate back
driver.refresh()	Refresh page
driver.quit()	Close browser

11. Expected Output

Python script executes successfully, Chrome browser opens automatically, the target webpage loads, browser title is displayed, and the browser closes without errors.

12. Advantages

- Open-source and free
- Supports multiple browsers
- Supports multiple programming languages
- Easy CI/CD integration
- Reusable automation scripts

13. Limitations

- Limited to web application testing
- Dynamic web pages may require advanced locators
- Requires browser compatibility

14. Viva Questions

1. What is Selenium?

2. What is Selenium WebDriver?

3. What is Selenium Grid?

4. What is Selenium IDE?

5. What are Selenium locators?

6. What is ChromeDriver?

7. What is Continuous Testing?

8. How does Selenium integrate with Jenkins?

9. What is cross-browser testing?

10. Why is Selenium important in DevOps?

15. Examination Questions

- Define Selenium.
- What is Selenium WebDriver?
- What is a Locator?
- Explain Selenium architecture.
- Explain Selenium installation and configuration.
- Explain Selenium WebDriver architecture and write a program for automated browser testing.
- Discuss the role of Selenium in DevOps and Continuous Testing.

16. Result

Selenium was successfully installed and configured. A basic Selenium automation script was executed successfully, demonstrating automated browser testing and validating Selenium's role in DevOps.

DEVOPS LAB MANUAL

Experiment No. 11
Title: Write a Simple Program in JavaScript and Perform Testing using Selenium

1. Aim of the Experiment
To develop a simple JavaScript-based web application and automate its testing using Selenium WebDriver.

2. Objectives
- Create a simple JavaScript web application.
- Understand JavaScript event handling.
- Install and configure Selenium.
- Automate browser interactions using Selenium WebDriver.
- Verify application output automatically.
- Understand the role of Selenium in Continuous Testing and DevOps.

3. Apparatus / Software Required
- Windows/Linux/macOS
- Python 3.x
- Google Chrome Browser
- Selenium Library
- Visual Studio Code
- Live Server Extension
- Internet Connection (for installation)

4. Prerequisites
- Basic knowledge of HTML, CSS and JavaScript.
- Basic Python programming knowledge.
- Selenium installed using pip.
- Chrome browser available.

5. Theory
JavaScript is a client-side scripting language used to create dynamic web pages. Selenium WebDriver automates browser actions and validates application behavior without manual

intervention. In DevOps, Selenium is integrated into CI/CD pipelines to perform Continuous Testing whenever code changes are committed.

6. Real-Time Applications

- Amazon: Automated testing of login, search, shopping cart and checkout pages.
- Flipkart: Automated regression testing before Big Billion Days releases.
- IRCTC: Automated validation of login, ticket booking and payment workflows.
- Banking applications: Automated testing of online transactions and account services.

7. Sample JavaScript Application (HTML)

```
<!DOCTYPE html>
<html>
<head>
<title>JavaScript Demo</title>
</head>
<body>

<h2>Addition Program</h2>

<input id="num1" placeholder="First Number">
<input id="num2" placeholder="Second Number">

<button onclick="addNumbers()">Add</button>

<p id="result"></p>

<script>
function addNumbers(){
let a = parseInt(document.getElementById("num1").value);
let b = parseInt(document.getElementById("num2").value);
document.getElementById("result").innerHTML = "Sum = " +
(a+b);
}
</script>

</body>
</html>
```

8. Selenium Python Program

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get("http://127.0.0.1:5500/index.html")

driver.find_element(By.ID,"num1").send_keys("25")
driver.find_element(By.ID,"num2").send_keys("15")

driver.find_element(By.TAG_NAME,"button").click()

time.sleep(2)

result = driver.find_element(By.ID,"result").text

print(result)

driver.quit()
```

---

9. Algorithm

Step 1: Create the HTML page with JavaScript.

Step 2: Run the page using Live Server.

Step 3: Install Selenium.

Step 4: Open Chrome browser using Selenium.

Step 5: Load the web page.

Step 6: Enter two numbers.

Step 7: Click the Add button.

Step 8: Read the displayed result.

Step 9: Verify the output.

Step 10: Close the browser.

10. Expected Output

The browser opens automatically, the numbers are entered, the Add button is clicked, and the application displays 'Sum = 40'. Selenium successfully verifies the result.

11. Advantages

- Automates repetitive testing.
- Improves software quality.
- Supports multiple browsers.
- Easy integration with Jenkins CI/CD.
- Reduces manual testing effort.

12. Limitations

- Limited to web applications.
- Dynamic elements may require advanced locators.
- Browser compatibility must be maintained.

13. Viva Questions

1. What is JavaScript?

2. What is Selenium?

3. What is Selenium WebDriver?

4. What is an event in JavaScript?

5. What is the purpose of onclick()?

6. What is send_keys()?

7. What is click()?

8. What is By.ID?

9. Difference between manual and automation testing?

10. How does Selenium support DevOps?

14. Examination Questions

- Define JavaScript.
- Define Selenium.
- What is Selenium WebDriver?

- Explain Selenium automation workflow.
- Explain JavaScript event handling.
- Develop a JavaScript program and automate its testing using Selenium with suitable explanation.
- Explain the role of Selenium in DevOps Continuous Testing.

15. Result

The JavaScript application was successfully developed and tested using Selenium WebDriver. The automated test validated the application output, demonstrating the use of Selenium for Continuous Testing in DevOps.

DEVOPS LAB MANUAL

Experiment No. 12
Title: Develop Test Cases for the Containerized Application using Selenium

1. Aim of the Experiment
To design and execute automated Selenium test cases for the Docker containerized web application developed in Experiment 7 and deployed using Docker/Kubernetes.

2. Objectives
- Understand test case design for web applications.
- Develop Selenium automated test scripts.
- Validate functionality of a containerized application.
- Verify application behavior after deployment.
- Understand Continuous Testing in a DevOps pipeline.

3. Apparatus / Software Required
- Windows/Linux/macOS
- Python 3.x
- Selenium Library
- Google Chrome
- Docker Desktop
- Containerized application from Experiment 7
- Kubernetes (optional, if deployed via Exp. 9)
- Visual Studio Code

4. Prerequisites
- Experiment 7 completed.
- Application running in Docker (or Kubernetes).
- Python and Selenium installed.
- Chrome browser available.

5. Theory
Automated testing verifies application functionality without manual intervention. In DevOps, Selenium is integrated with CI/CD pipelines to validate every deployment. When applications are containerized with Docker and orchestrated by Kubernetes,

Selenium helps ensure that the deployed application behaves correctly in production-like environments.

6. Real-Time Applications

- Amazon: Automated UI tests validate shopping cart and checkout after every deployment.
- Flipkart: Regression test suites execute before festival-sale releases.
- IRCTC: Login, booking and payment workflows are validated automatically.
- Banking Portals: Customer transactions are tested before production rollout.

7. Sample Test Cases

TC ID	Test Scenario	Expected Result	Status
TC01	Application loads successfully	Home page displayed	Pass
TC02	Verify page title	Correct title displayed	Pass
TC03	Enter valid input	Data accepted	Pass
TC04	Click Submit/Register	Success message displayed	Pass
TC05	Refresh application	Application reloads successfully	Pass
TC06	Verify deployed container URL	Accessible through browser	Pass

8. Sample Selenium Program

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("http://localhost:8080")
print("Title:", driver.title)
# Add element interactions as needed
driver.quit()
```

9. Algorithm

Step 1: Start the Docker/Kubernetes application.

Step 2: Launch Chrome using Selenium.

Step 3: Open the application URL.

Step 4: Execute test cases.

Step 5: Verify expected results.

Step 6: Record pass/fail status.

Step 7: Close browser.

10. Expected Output

The containerized application opens successfully. All defined Selenium test cases execute successfully and the expected results are obtained.

11. Advantages

- Automates repetitive testing
- Improves deployment confidence
- Supports regression testing
- Integrates with Jenkins CI/CD
- Reduces manual effort

12. Limitations

- UI changes require script maintenance
- Dynamic pages may need advanced waits
- Browser compatibility should be managed

13. Viva Questions

1. What is a test case?

2. Why use Selenium for containerized applications?

3. What is regression testing?

4. What is Continuous Testing?

5. How does Docker support testing?

6. How does Kubernetes help deployment?

7. What is an assertion?

8. What is expected result?

9. What is test automation?

10. How is Selenium integrated with Jenkins?

14. Examination Questions

- Define Test Case.
- What is Selenium?
- What is Regression Testing?
- Explain Selenium test case execution.
- Explain automated testing for containerized applications.
- Develop Selenium test cases for a Docker containerized application and explain the execution workflow.

15. Result

Test cases were successfully designed and executed for the containerized application using Selenium. The automated tests validated the application's functionality, demonstrating the role of Continuous Testing in a DevOps environment.